Pioneer.jl SearchDIA — Allocation Attribution

Current state, per-phase / per-function / per-line breakdown, and next targets

2026-06-24 | branch `perf/runtime-profiling` @ `3651b1da9`

TL;DR

The allocation-reduction effort has cut warm 2-file MTAC total allocation **146.94 GB → 58.10 GB (−60%)** and runtime **101.95 s → 91.30 s (−10%)**, with results bit-identical. The two heavily-optimized phases (Main Search, Precursor Scoring) are no longer dwarfing everything, which *promotes* three previously-ignored phases — **Quantification & Output, Protein Scoring, Spectral Library Loading** — to the top of the remaining list. The two largest reusable per-thread buffers (fragment-index scratch, scored-PSM buffer) are *already* reused across files; they are not the remaining allocators. The best next targets are the **serial O(library) map-builds** in Quant & Output and Protein Scoring (∼3.9 GB combined, single-threaded, so also a latency win).

1 Methodology & confidence

Two independent measurements, used for different things:

- **Phase magnitudes** — Pioneer's own per-phase accounting (`Mem Alloc (GB)` column), warm run (2nd `SearchDIA` in-process), 2-file MTAC_3P, 8 threads. This is the *ground truth for magnitude*.
- **Within-phase line/function ranking** — `Profile.Allocs` sampling profiler, single-thread, 1-file MTAC, sample rate 2×10^{-4}. Reliable for *which* lines allocate; absolute GB trustworthy only for high-sample-count lines. Single-thread totals do not match the 8-thread phase table and must not be read as magnitudes.

Caveat (read before trusting any single line): the sampling profiler's top two leaf frames — `process_scans_fused.jl:131` (92 GB from *7* samples) and `scoring.jl:1459 select_best_per_precursor!` (22 GB from *1* sample) — are low-sample, high-variance extrapolations. They correctly *point* at the per-scan fused deconv/score loop (Main Search) and best-PSM selection (Precursor Scoring) as hotspots, but their GB figures are noise: the phase table caps Precursor Scoring at 7.4 GB total, so "22 GB" cannot be real. Magnitudes below are anchored to the phase table, not these extrapolations.

Update — Main Search resolved by deterministic bucketing. The `process_scans_fused.jl:131` mystery was settled with `gc_bytes()` sub-attribution (env `PIONEER_DIAG_MSLOOP`, single-thread, 1-file MTAC, 4,612,759 PSMs) — see the Main Search subsection. The 92 GB extrapolation was an artifact; the real in-loop total is ∼3.8 GB, dominated by one-time buffer-growth churn, not the deconvolution.

2 Phase-level attribution (warm, 2-file MTAC, 8 threads)

Phase	Baseline (GB)	Current (GB)	Δ	Status
Spectral Library Loading	4.16	4.16	0%	untouched
Parameter Tuning	3.37	1.50	−55%	optimized
Quadrupole Tuning	0.20	0.18	–	small
BitVec Calibration	6.08	1.05	−83%	optimized
Main Search	59.64	29.43	−51%	**still #1**
Precursor Scoring	58.90	7.37	−87%	optimized
Huber Calibration	0.30	0.30	–	small
Chromatogram Integration	3.56	3.36	−6%	lightly touched
Protein Inference	0.48	0.48	–	small
Protein Scoring	4.19	4.20	0%	**untouched**
Quantification & Output	5.97	5.97	0%	**untouched**
TOTAL	**146.94**	**58.10**	−60%	
Runtime (s)	**101.95**	**91.30**	−10%	

"Current" = combined-allocations state. The just-landed `est_per_thread/n_threads` commit (`3651b1da9`) trims Main-Search scratch further — measured −3.45 GB on 3-file SCP (where it matters most), ∼ −0.35 GB on this MTAC config — not yet folded into the table above.

3 Within-phase function / line attribution

Ranked by reliable (high-sample) contribution; phase magnitude in brackets.

Main Search [29.4 GB] — sub-attributed via `gc_bytes` bucketing

Deterministic per-sub-step measurement of the fused scan loop (single-thread, 1-file MTAC, 4,612,759 PSMs). This replaces the unreliable 7-sample profiler frame. In-loop total ∼3.8 GB:

Sub-step	GB	Nature	Scales with
`score` (`score_psms!`)	2.752	one-time buffer-growth churn	run (amortizes)
`df_materialize`	0.778	per-file output DataFrame	files
`collect` (`sub_indices`)	0.204	per-scan temp Vector **[FIXED]**	scans × files
`post_dm`	0.077	per-scan, small	scans
`prep`	0.019	per-scan, small	scans
`run_fused` / `dist` / `reset`	∼0.001	negligible	—

- **`score` = 2.75 GB is `growScoredPSMs!` block(500k) reallocation churn** — copying the ∼46-field struct array as it grows 5k → 4.6M on the *first* file (the ∼2.5× copying overhead). Buffer capacity is retained across files, so this is **one-time per run** and amortizes to nothing on multi-file runs. Consistent with keeping block(500k).
- **`collect` = 0.20 GB — FIXED** (`31d334c63`). `sub_indices = collect(Int64, prec_range)` allocated a fresh `Vector` every scan; `prec_range` is already a `UnitRange{Int64}` and `run_fused!` only iterates it, so it was pure waste. Now passed directly via type-assert. Per-scan ⇒ scales with scans×files (∼4 GB on a 20-file run); output byte-identical, PSM count unchanged.

- **The deconvolution itself barely allocates** (`run_fused`, `dist`, `post_dm` ≈ 0). The loop is already lean — the big numbers are one-time growth + unavoidable per-file output.

Profiler residuals also seen (smaller, partly compile-time in the single-thread profile): `features.jl` per-scan feature-row writes (∼0.55 GB), `_ms1_lookup_scan_runs!` (∼0.22 GB), `irt_refinement.jl` `count_token_ids!` (0.15 GB).

Precursor Scoring [7.4 GB]

- `scoring.jl:1459` `select_best_per_precursor!` — best-PSM selection (sort/group). Profiler over-extrapolates (1 sample); real cost bounded by the 7.4 GB phase total.
- `wide_window_features.jl` (`:99/461/558/840/844`) — ∼0.38 GB, flanking-window feature construction.

Quantification & Output [6.0 GB — untouched]

- `maxLFQ.jl:653--660` `build_accession_to_species` — **∼2.3 GB.** Serial O(library) accession→species map rebuilt during output. High sample count ⇒ trustworthy. **Top easy target.**
- `MergeOperations.jl:67` `fillColumn!` — 0.18 GB; merge/output column fills.

Protein Scoring [4.2 GB — untouched]

- `run_protein_scoring.jl:39/44/48` `count_protein_peptides` — **∼1.6 GB.** Serial O(library) peptide-per-protein counting. High sample count ⇒ trustworthy. **Second easy target.**

Chromatogram Integration [3.4 GB] & Spectral Library Loading [4.2 GB]

- `utils.jl:854` `build_chromatograms` (0.15 GB), `integrate_chrom.jl:273/279` `WHSmooth!` (∼0.11 GB).
- Library loading is dominated by `serialization.jl` deserialize (∼1.2 GB sampled) — one-time, hard to avoid.

4 Reusable-buffer status (answered)

Buffer	Reused across files?	Mechanism
Frag-index emit scratch	**Yes**	`SearchContext.frag_index_scratch`, grow-only `prepare!`
Per-thread counters/int bufs	**Yes**	same `FragIndexScratch`
Scored-PSM buffer	**Yes**	`SearchData.main_search_scored_psms`; alloc once (5000), grow-only, capacity retained

Both large per-thread buffers live on the per-run `SearchContext` and are grown once to a high-water-mark, then reused with zero further allocation (Julia never shrinks array capacity on `resize!`-down). The residual Main-Search allocation is *not* these buffers — it is the per-file *materialization* of PSM rows (the inner `vcat` that decouples the result from the reused buffer before the next file overwrites it), which is legitimate output, plus the per-scan work in the fused loop.

5 Recommended next targets (priority order)

1. **[DONE `31d334c63`] Main Search `collect` (∼0.2 GB/file).** Per-scan `Vector` removed; bit-identical. The only clearly-wasteful per-scan allocation in the fused loop.

2. `build_accession_to_species` **(∼2.3 GB, Quant & Output).** Serial, O(library), rebuilt at output time. Build once and cache on the spec-lib / search context, or restructure to avoid the per-call dict. Also a single-threaded latency win. *One-time per run — amortizes across files.*

3. `count_protein_peptides` **(∼1.6 GB, Protein Scoring).** Same shape — serial O(library) count. Precompute/reuse. *Also one-time per run.*

Diminishing returns ahead. With `collect` fixed, the Main Search per-scan path is lean — its remaining allocation is one-time buffer growth (`score`, 2.75 GB, amortizes) plus unavoidable per-file output (`df_materialize`, 0.78 GB). The `StandardLibraryPrecursors` constructor (∼0.8 GB) was investigated and found to be inherent one-time Arrow string materialization over 6.8 M precursors (the getters themselves do *not* allocate). So the remaining targets (2–3) are all O(library) *one-time* costs that look large on 1–2 file profiling configs but amortize on production multi-file runs. The allocation that genuinely scales with dataset size — the per-file Main Search loop — has been brought to near-minimal.